SLE-4: Architecture Decision Record (ADR)

Graph Search System Using BFS and DFS

PRN: 25UAM098
Name: Akhila Kiran Dhutre
Division: B
Date: 08/10/2026
Status: Accepted

Part A — Architecture Decision Record (ADR)

ADR-01: Use Iterative DFS with an Explicit Stack Instead of Recursion

1. Context

The project implements Breadth-First Search (BFS) and Depth-First Search (DFS) for graph traversal. BFS uses a queue, while the DFS implementation in this project is explicitly iterative and uses a stack. The DFS code contains no recursive function calls. The design therefore requires an explicit mechanism to manage the order in which DFS nodes are explored.

2. Decision

I decided to implement DFS using an **explicit stack with an iterative approach** instead of using recursive DFS. The stack stores nodes that still need to be explored, and the algorithm repeatedly removes a node from the stack and processes its neighbours.

3. Alternatives Considered

Alternative 1: Recursive DFS
Recursive DFS is a standard approach, but it depends on the function call stack. I did not choose it because this project uses an iterative DFS implementation and an explicit stack gives direct control over traversal state.

Alternative 2: Iterative DFS using an explicit stack
This was selected because it makes the stack operation visible in the implementation and avoids dependence on recursive function calls.

Alternative 3: Use BFS-style queue processing for DFS
This was rejected because a queue follows FIFO order and produces breadth-first behaviour. DFS requires stack-based LIFO processing to explore a path deeply before backtracking.

4. Rationale / Justification

The selected design directly matches the DFS implementation used in the project. The DFS function is iterative and creates an explicit stack:

```
visited = set()
stack = [start]
```

During traversal, the algorithm removes a node using **stack.pop()**. It then examines the neighbours and adds unvisited neighbours to the stack. Reversing the neighbour order helps maintain a predictable traversal order.

The explicit stack makes the DFS traversal mechanism easy to see, test and explain. It also avoids recursive function calls, making the implementation independent of the program's recursion depth.

The decision is supported by the SLE-2 experiment. In the recorded 20-node graph experiment, DFS was executed for **50,000 repetitions**. Its three recorded runs were 186.3295 ms, 223.1545 ms and 187.1891 ms, giving an average of **198.8910 ms**. DFS expanded **16 nodes** in one search.

The measured result is specific to the graph, implementation and execution environment. The ADR therefore does not claim that iterative DFS is always faster than recursive DFS. The main reason for the decision is the project's need for an explicit, non-recursive DFS implementation with direct control over the traversal stack.

5. Consequences

Positive Consequences

- Avoids recursive function calls in the DFS implementation.
- Makes the DFS stack explicitly visible in the program.
- Provides direct control over the order of node exploration.
- Avoids dependence on the recursion depth of the Python interpreter.
- Makes the implementation easy to connect with the stack-based definition of DFS.
- The design can be profiled and measured as part of the existing SLE-2 experiment.

Negative / Trade-offs

- The programmer must explicitly manage the stack.
- The traversal logic is more manual than a short recursive DFS implementation.
- Neighbour ordering must be handled carefully when a specific traversal order is required.
- The explicit stack still requires memory proportional to the traversal state.

Part B — Short SLE Journey

SLE-1

In SLE-1, I worked on the initial graph search implementation and developed the BFS and DFS search logic. I learned how different graph traversal strategies work and how appropriate data structures support graph traversal.

SLE-2

In SLE-2, I compared BFS and DFS performance using the same graph conditions. The experiment used a 20-node graph, 50,000 repetitions and repeated timing runs. BFS had an average execution time of 136.2527 ms, while DFS had an average execution time of 198.8910 ms. BFS expanded 20 nodes and DFS expanded 16 nodes. PySpy profiling was also attempted.

SLE-3

In SLE-3, I designed the C4 architecture for the BFS/DFS Graph Search Performance Analysis System. The architecture included the Input / Graph Module, Search Engine, Visited / Memory Module, Performance Measurement Module and Output Module. The Search Engine contains BFS and DFS, with BFS using a queue and DFS using a stack.

SLE-4

In SLE-4, I selected the implementation decision of using an explicit stack for iterative DFS instead of recursion. I documented the alternatives, reasons, benefits and trade-offs of this decision using an Architecture Decision Record.

Part C — AI Contribution

AI tools used: **ChatGPT**

What AI helped with:

- Understanding the SLE-4 ADR requirements.
- Structuring the Architecture Decision Record.
- Explaining the iterative DFS and explicit-stack design decision.
- Organizing the SLE-1 to SLE-4 journey.
- Improving the clarity of the technical explanation.

What I did myself:

- Developed and validated the BFS and DFS implementation.
- Implemented DFS iteratively using an explicit stack.
- Created the graph configurations used for the experiments.
- Conducted the performance experiments.
- Collected and verified the experimental results.
- Performed the profiling work using PySpy.
- Designed the C4 diagrams using draw.io.
- Reviewed and understood the design decisions used in the project.

Conclusion

The important architecture decision in my project is to implement DFS iteratively using an **explicit stack instead of recursion**. This decision directly matches my actual DFS code, where a stack is created and managed using stack operations.

The decision gives direct control over DFS traversal and avoids recursive function calls. It also clearly represents the stack-based nature of Depth-First Search.

The SLE-2 experiment provided practical evidence about the implemented DFS. For 50,000 repetitions on the recorded 20-node graph, DFS had an average execution time of **198.8910 ms** and expanded **16 nodes** in one search.

Overall, SLE-4 helped me identify, document, justify and defend a design decision that is specific to my own BFS/DFS implementation.